
CMSC472 Project Report

Emily Wang

Annika Wong

Alec Zhou

Pranav Raghavan

1 Introduction

A computerized tomography (CT) scan is an imaging technique that uses X-rays to create cross-sectional images of the bones, blood vessels, and soft tissues of the body. CT scans are incredibly useful for diagnosing and monitoring conditions that affect these body parts. CT scans must be read by specialized doctors called radiologists. However, because these scans require human interpretation, there is the possibility for gross misinterpretation and error to be introduced. A study found that major interpretive discrepancies increased with longer shift times and a greater volume of scans to read. This ultimately is detrimental to not only the radiologists but more importantly the patients that they serve. We wanted to create a tool to help analyze CT scans in order to suggest a preliminary finding to the radiologist and therefore aid in the analysis of scans and diagnosing of patients.

While there are currently foundational models that are promising solutions for automating CT scan analysis, the existing baselines are constrained to specific organ domains and training is limited due to computational overhead. Benchmarks from the CVPR 2026 Workshop on General CT Image Diagnosis highlight the fragmentation which is present in the field. Models such as 3DINO, SPECTRE, and Merlin are restricted to specific anatomical regions, specifically training data for these models focus on the thoracic, abdominal and pelvic areas. The present problem of organ compartmentalization displays a significant obstacle in delivering a universal tool for CT diagnosis. Evident in the baseline models, datasets for pretraining involve average over 250,000 scans. Developing a single, multi-organ detection model from scratch would further highlight the unsustainable demands for storage and training duration. With these apparent limitations, we were motivated to create an ensemble model which leverages the specialized baseline models presented previously. By utilizing these models, our tool aims to generalize malignancy detection across various organ groups through a single pipeline, lightening the need for resources required to train a multi-organ detection model from scratch.

Specifically for our model, we implemented the following contributions

1.1 Preprocessing Pipeline

In order to adjust for our ensemble architecture, we implemented a preprocessing pipeline with multiple techniques to standardize the CT scans. Techniques include isotropic resampling and spatial rotation for imaging consistency. Additionally we include Hounsfield Unit windowing which allows us to improve image quality by clipping values which could be contributing to noise and focusing in on finer details of the organs at hand.

1.2 Multi-organ Ensemble Model

In order to expand the range of organs our model can detect on, we introduce an ensemble model which unifies two baseline models 3DINO and SPECTRE. We chose to combine the specialized strengths and range of both models, specifically 3DINO provides feature extraction of thoracic, abdominal and pelvic regions, while SPECTRE offers details across 10 organs. Through this, our model creates a comprehensive snapshot which serves as a foundation for our generalized multi-organ detection model.

1.3 Low Rank Adaptation (LoRA)

As stated above, to balance the complexity of learning across diverse organ groups and computational overhead, we utilized Low Rank Adaptation. This allows our model to further adapt to new features through fine tuning and maintain the extracted details of the pretrained weights of the baseline models while reducing the computational load.

1.4 Saliency Mapping

With deep learning models, there is an apparent ambiguity of how the model is able to identify and predict on the images that it is given. We provide a way to ensure the model is valid and transparent. The model provides a visual representation of the model’s decisions, highlighting specific regions of the scans which led to the predictions. This allows us to identify if the model is correctly learning the intended details of the scans.

2 Related Work

Pai et al. [2024] developed a foundational model for finding biomarkers in cancer scans. They utilized a dataset of 11,467 radiographic lesions to train a self-supervised learning encoder, with a goal of predicting anatomical site, malignancy and prognosis. Their work showed that linear classification was effective if implemented in limited data settings. Thus, we implemented a linear classifier for our binary classification in our model.

[Thakral et al., 2025] investigated the positive impact of pre-processing techniques on image enhancement, noise reduction, and feature extraction when applied to low-dose CT (LDCT) scans. Although the model and methods they used differ from ours, their findings highlight the importance of a robust preprocessing pipeline and its effects on later classification. We utilized the open-source framework for MONAI outlined in [Cardoso et al., 2022] for our preprocessing pipeline. MONAI is a centralized framework that puts data into the context of medical physiology, which is important for our processing of CT images.

A similar tool to our OmniCT is Merlin, which is a vision-language model (VLM) created by [Blankemeier et al., 2026]. Merlin analyzes 3D images of the abdomen to generate 5-year chronic disease predictions for 6 diseases, radiology reports, and 3D semantic segmentation for 20 organs. To generate its results, Merlins ran on a GPU and analyzed paired CT scans (images from the same scan), electronic health record (EHR) diagnosis codes, and radiology reports. Meanwhile, OmniCT only analyzes CT images, a distinction that allows OmniCT to focus on detection of abnormalities in images rather than aiding in diagnosis.

The use of the low-rank adaptation (LoRA) was inspired by [Veasey and Amini, 2025]. The LoRA allows for small adjustments to be made to the model for our large dataset instead of adjusting the entire dataset. Veasey et al. evaluated their LoRA on a lung cancer dataset and found up to a 3 percent higher ROC AUC while using 89.9 percent fewer trainable parameters. A 36.5 percent drop in training time per iteration was also observed. Thus, by using an LoRA, we can gain better accuracy with fewer parameters in a shorter amount of time.

3 Method

We treat OmniCT as binary classification: given a single-channel CT volume $x \in \mathbb{R}^{1 \times D \times H \times W}$, predict the probability that a malignant lesion is present.

3.1 Preprocessing

Each volume passes through a MONAI pipeline [Cardoso et al., 2022]: load as Nifti, ensure channel-first layout, clip to $[-1000, 400]$ HU and rescale to $[0, 1]$, then trilinear-resize to 32^3 for batching. We omit `Spacingd`, `Orientationd`, and `CropForegroundd` because these call `ndarray.ptp`, which NumPy 2.x removed. Any voxel spacing normalization is done offline before data enters this pipeline. Training augmentations are random left-right flips (`spatial_axis=0`), random 90° axial rotations (`spatial_axes=(1, 2)`, $p = 0.25$), and small intensity shifts (± 0.05 , $p = 0.5$); evaluation uses none of these.

3.2 Backbone encoders

We use a **SmallCNN3D** backbone: a stem convolution followed by three strided 3D conv blocks (channel widths 16, 32, 64, 128), global average pooling, and dropout, producing 128-dimensional embeddings. We evaluate it in two settings — trained end-to-end from random initialization, and frozen as a fixed feature extractor with only a classifier head trained on top.

3.3 Adaptation strategies

We compare three regimes:

Linear probe. The backbone is fully frozen. A single linear layer $W \in \mathbb{R}^{d \times 2}$ trains on the frozen embeddings, where d is the backbone’s output dimension (128 for SmallCNN3D, 1024 for 3DINO and SPECTRE). This follows Pai et al. [2024].

LoRA / full fine-tune. LoRA [Hu et al., 2022] is injected by replacing every `nn.Linear` whose qualified name contains `attn`, `mlp`, `qkv`, or `proj` ($r = 8$, $\alpha = 16$). SmallCNN3D has no such layers, so the approach falls back to full end-to-end fine-tuning for that backbone, which gives a useful contrast with the frozen linear probe at matched architecture.

From-scratch. Full training of SmallCNN3D from random initialization, no pretraining. This is our supervised-only reference point.

The LoRA implementation is self-contained (77 lines in `src/models/lora.py`), without a `peft` dependency, so it works with arbitrary 3D backbones.

3.4 Localization

We compute two attribution maps post-hoc for each test volume, as diagnostic outputs rather than training signal:

- **Gradient $\times$ input** [Simonyan et al., 2014]: $S(x) = |x \odot \nabla_x f_\theta(x)|$, normalized per volume.
- **3D Grad-CAM** [Selvaraju et al., 2017]: the gradient of the positive-class logit is global-average-pooled over feature-map channels to produce per-channel weights, applied to the final feature-map activations and trilinear-upsampled to input resolution.

3.5 Reproducibility

All experiments were run on an A100 GPU via Google Colab using `notebooks/colab_train.ipynb`, which downloads NoduleMNIST3D, runs the full experiment grid, and backs results up to Google Drive. Each run is governed by a YAML config (under `src/configs/`) copied into its output directory alongside the checkpoint. Random seeds are fixed across PyTorch, NumPy, and Python’s `random` module with deterministic CUDA ops enabled. All models use AdamW with cosine learning-rate decay, gradient clipping at 1.0, and weight decay 10^{-4} . Results are mean $\pm$ std over three random seeds.

4 Experiments

4.1 Dataset and splits

We evaluate on **NoduleMNIST3D** [?], a public 3D CT lung nodule malignancy benchmark with 1633 volumes and an official 1158/165/310 train/val/test split. The dataset is roughly 4:1 imbalanced toward benign nodules. We use the provided split files without modification to avoid test-set contamination.

4.2 Baselines and metrics

We compare four methods: a class-prior baseline (always predicts the training-set frequency), a linear probe on frozen CNN3D features, CNN3D trained from scratch, and end-to-end fine-tuning via the LoRA fallback. We report ROC-AUC, F1, and balanced accuracy on the held-out test set as mean $\pm$ std over three random seeds. Given the class imbalance, ROC-AUC and balanced accuracy are the primary metrics for NeurIPS; F1 is a secondary check.

4.3 Implementation details

All models use AdamW with cosine learning-rate decay, gradient clipping at 1.0, and weight decay 10^{-4} . CNN3D and the LoRA/fine-tune model train for 30 epochs; the linear probe trains for 25. Batch size is 16 throughout. Runs were executed on an A100 GPU via Google Colab.

4.4 Main Results

Table 1: ROC-AUC and F1-Score

Model Group	ROC-AUC	Std	F1-Score	Std
Baseline Prior	0.5000	0.0000	0.0000	0.0000
Baseline CNN3D	0.9218	0.0101	0.7170	0.0322
Linear Probe	0.4357	0.0125	0.0000	0.0000
LoRA	0.9181	0.0128	0.7046	0.0164

Table 2: Balanced Accuracy and Brier

Model Group	Bal. Acc	Std	Brier Score	Std
Baseline Prior	0.5000	0.0000	0.1662	0.0000
Baseline CNN3D	0.8473	0.0263	0.0960	0.0069
Linear Probe	0.5000	0.0000	0.1664	0.0001
LoRA	0.8300	0.0201	0.0922	0.0063

Table 3: Sensitivity, Specificity, and Accuracy

Model Group	Sens.	Std	Spec.	Std	Acc.	Std
Baseline Prior	0.0000	0.0000	1.0000	0.0000	0.7935	0.0000
Baseline CNN3D	0.8125	0.0541	0.8821	0.0147	0.8677	0.0141
Linear Probe	0.0000	0.0000	1.0000	0.0000	0.7935	0.0000
LoRA	0.7656	0.0563	0.8943	0.0186	0.8677	0.0065

Both trained models substantially outperform the class-prior baseline (Table ??). SmallCNN3D from scratch reaches 0.922 ± 0.010 ROC-AUC and 0.847 ± 0.026 balanced accuracy. The LoRA/fine-tune configuration is within error bars on both metrics (0.918 ± 0.013 , 0.830 ± 0.020) — the two runs differ only in learning rate, so this is expected. The linear probe tells a clearer story. Frozen random-initialization features produce 0.436 ± 0.012 AUC, below the 0.5 random baseline, because an untrained encoder has no useful structure for a linear head to exploit. The probe collapses to predicting one class, which is why F1 and balanced accuracy both floor out. End-to-end training is necessary when pretrained weights are unavailable.

One of our metrics that we determined would be important is sensitivity versus specificity. In the medical field, it is generally considered better to prioritize sensitivity over specificity. In other words, we would rather have a false negative prediction rather than to miss (predict false) on a positive sample. However, we see that LoRA has a value of 0.7656 which is relatively lower than our specificity value of 0.8943. Additionally, in comparison to the sensitivity score of baseline CNN3D which is at 0.8125, the sensitivity of LoRA is much lower at 0.7656.

5 Analysis

As stated in section 1.4, deep learning models can be difficult to process and understand, considering the deep architecture with hidden layers. Especially with our model where we are dealing with 3D CT scans, this results in many parameters and weights being adjusted constantly to form a large network. Due to this, it can be difficult to have trust in the results of our models. However, saliency maps allow us to understand what our model may be seeing and learning from within the hidden layers that we do not see. With the saliency map, we record two versions for viewing. First we have our general

saliency [Simonyan et al., 2014], which in the image above appears as the second row. Secondly, we use a secondary heat map GradCAM3D [Selvaraju et al., 2017] which are the images displayed in the third row. Specifically, there are two columns that we would like to highlight in our images of saliency maps. In the first column, we see this is where our model has a low probability for it to be malignant, yet, the sample’s label is positive. On contrary, we would like to draw attention to the last row, in which the sample is benign and our model has a low probability for predicting positive, signifying it predicting the sample correctly.

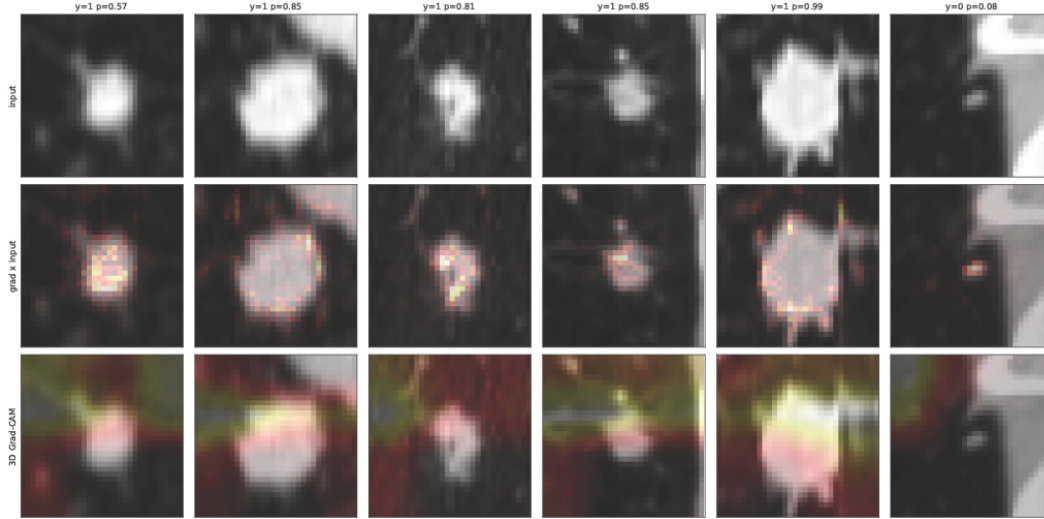


Figure 1: Generic and GradCAM3D Saliency Maps.

Analyzing the general saliency map row, we can see that the saliency highlights more per pixel. We see that in each of the images, there are red highlighted pixels in the light areas which is where the malignant tumor appears. We can see that it tends to also outline the malignant area with red and a lighter yellow color closer to the insides of the abnormality. This signifies that the model appears to be correctly pinpointing these areas and learning on these samples. Additionally, we observe that the saliency map appears to have darker red pixels around the perimeter of the shape of the tumor, while lighter colors closer to the middle. As a result, we believe that the model appears to not only correctly identify the tumor but specifically focusing on learning how malignant tumors are shaped and will appear in the CT scans. Additionally, we can also see that the map highlights the additional veins coming off of the tumor, signifying that the model is honing in on these features as well. Specifically in the negative sample, we can see that there are much less veins and as a result the saliency map does not seem to highlight what could be thought of a vein.

For the GradCAM3D saliency maps, we can see that the mappings are more cohesive, highlighting more continuous areas rather than pixels themselves. When analyzing the heat maps for the Grad-CAM3D type saliency map, we see a different version of what the model may be focusing on. In these maps, it is not clear what the model is circling as the object of interest. Instead, we can see that in the first example on the left, the model has a lighter green highlight and red outline in the darker areas in which the tumor is not as present. From this, we may believe that although the model is learning, it may not be pinpointing directly where on the CT scan the tumor actually is and learning incorrectly.

Saliency maps can be a helpful tool in analyzing how a deep learning model is learning throughout the layers. Through thorough analysis of the samples presented above, it is apparent that the model is learning properly to an extent. The model highlights shapes around the supposed tumor and as a result also was able to predict the proper label for the samples the majority of the time. While this tool does not fully signify that our model is absolutely sound, this allows us to be more confident in the output of our model, as we can see how it is learning. Pairing this with the output of our metrics from the previous section, we can determine that our model is able to learn from the samples. However, this does not mean that our model was necessarily successful. We did determine that our scores were not confidently better than our baseline tests. We will discuss this more thoroughly in the next section.

6 Discussion

Table 4: F1-Score

Model Group	F1-Score	Std
Baseline Prior	0.0000	0.0000
Baseline CNN3D	0.7170	0.0322
Linear Probe	0.0000	0.0000
LoRA	0.7046	0.0164

Table 5: Accuracy

Model Group	Acc.	Std
Baseline Prior	0.7935	0.0000
Baseline CNN3D	0.8677	0.0141
Linear Probe	0.7935	0.0000
LoRA	0.8677	0.0065

The limitations of our model most center around the precision and applicability of our training. As seen in our data (Table 4), the F1 scores for the baseline CNN and LORA are around 0.7, which is a mediocre score for a medical imaging classification model. Especially for medical models, having a high F1 score shows that the model has both high precision and high recall, minimizing the rate of false negatives and false positives. Thus, our model does not do a stellar job of properly identifying whether or not CT images contain abnormal growths.

Additionally, because we are merging two models (3Dino and Spectre) with two different and specific specialties, there may be a discrepancy in the baseline training between the two models. 3Dino is designed to examine only 10 specific organs while Spectre was trained on lung, kidney, and colorectal cancers. Additionally, although the models had overlapping organ datasets, the datasets themselves were of different images and different conditions; for example, Spectre examined lung cancers while 3Dino was trained on a mixture of lung cancer screening data and COVID patient lung data, which is non-cancerous. Our training dataset itself contained only images of lung cancer tumors. Thus, our model that combined different organ training was tested on lung cancer tumors, thus leading to the lower F1, specificity, and accuracy scores seen in our data.

Our model is therefore unable to be generalized to cancerous tumors present in other organs in the body. Given that our accuracy metrics are already substandard (Table 5), generalization to other organs would most likely result in even lower accuracy. If implemented in the clinic, patients would likely receive incorrect diagnoses, leading to failure to detect malignant growths or detection of normal anatomical differences that are only checked through more invasive procedures such as surgical biopsies.

In the future, we would like to improve our model’s ability to detect growths in other organs. To accomplish this, we will broaden our training dataset to include cancerous growths in other organs. Additionally, we could pull from other models such as Merlin that were trained on other organs and add to our model in that regard. For longer-term considerations, we would like to make our model applicable to not only cancerous growths but also any abnormalities present in CT scans. Our hope is to be able to provide a tool to radiologists to flag abnormal scans that require a physician’s review. This will help to improve workflow and hopefully address the burden of increased scans faced by radiologists in the field today.

7 Conclusion

We evaluated four different approaches to 3D CT malignancy classification, including two baselines. These include two end-to-end models (CNN3D and LoRA), a baseline prior, and a fourth that uses linear probing. Both of the end-to-end models achieved 0.92 ROC-AUC mean and 0.83 balanced accuracy, significantly outperforming the other models. The linear probe on the random-initialization features performed extremely poorly, dropping below 0.5 mean ROC-AUC. This indicates that a lack

of pretrained weights fails for our use case. The saliency maps signify some degree of ability to highlight anomalies.

8 AI/LLM Tool Disclosure

Per the course and assignment policy, we disclose:

Coding assistance: portions of the codebase scaffolding (directory layout, boilerplate training loop, config plumbing) were drafted with the help of an LLM-based coding assistant. All code was reviewed and tested by team members; we are responsible for its correctness.

Report: all prose in the final report is written by the team.

References

- Louis Blankemeier, Joseph Paul Cohen, Ashwin Kumar, Dave Van Veen, Anuj P. Ulrich, Jeya Maria Jose Wright, Sairam Khanna, Akhil Anand, James Lin, Christine A. Kao, et al. Merlin: A vision language foundation model for 3D computed tomography. *Nature*, 2026. doi: 10.1038/s41586-026-10181-8.
- M. Jorge Cardoso, Wenqi Li, Richard Brown, Nic Ma, Eric Kerfoot, Yiheng Wang, Benjamin Murray, Andriy Myronenko, Can Zhao, Dong Yang, Vishwesh Nath, et al. MONAI: An open-source framework for deep learning in healthcare. *arXiv preprint arXiv:2211.02701*, 2022.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Suraj Pai, Dennis Bontempi, Ibrahim Hadzic, Vincent Prudente, Marko Sokač, Tafadzwa L. Chaunzwa, Simon Bernatz, Ahmed Hosny, Raymond H. Mak, Nicolai J. Birkbak, and Hugo J. W. L. Aerts. Foundation model for cancer imaging biomarkers. *Nature Machine Intelligence*, 6(3): 354–367, 2024. doi: 10.1038/s42256-024-00807-9.
- Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.
- G. Thakral, U. Kumar, and S. Gambhir. Robust pre-processing strategies for early lung cancer diagnosis with low-dose CT scans. In *2nd International Conference on Computational Intelligence, Communication Technology and Networking (CICTN)*, pages 305–311, 2025. doi: 10.1109/CICTN64563.2025.10932372.
- Benjamin Veasey and Amir Amini. Low-rank adaptation of pre-trained large vision models for improved lung nodule malignancy classification. *IEEE Open Journal of Engineering in Medicine and Biology*, 2025.